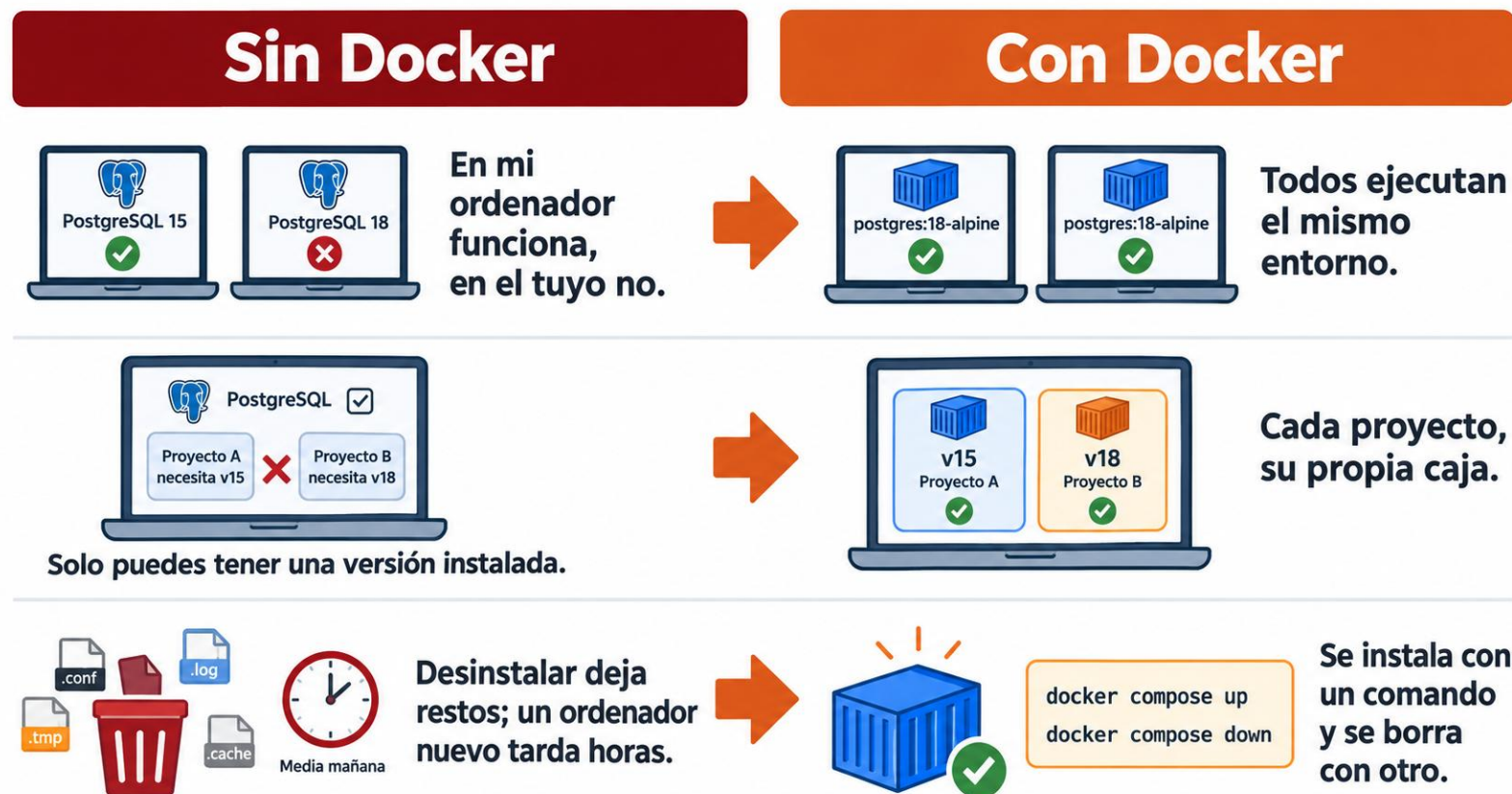


Introducción a Docker

Contenedores, imágenes, volúmenes y Docker Compose

Por qué Docker

Durante el curso vas a usar varios motores de bases de datos y herramientas con su propio servidor. Instalarlo todo en tu ordenador, con las versiones correctas y sin que choquen, es un dolor de cabeza. Docker resuelve exactamente eso.



¿Qué es Docker?

Docker ejecuta aplicaciones dentro de contenedores: entornos aislados y ligeros con todo lo que la aplicación necesita (código, dependencias, configuración), sin tocar el sistema operativo de tu máquina.

Aislado

Cada servicio vive en su propio contenedor y no se mezcla con los demás.

Ligero

Comparte el kernel del sistema: arranca en segundos y pesa poco.

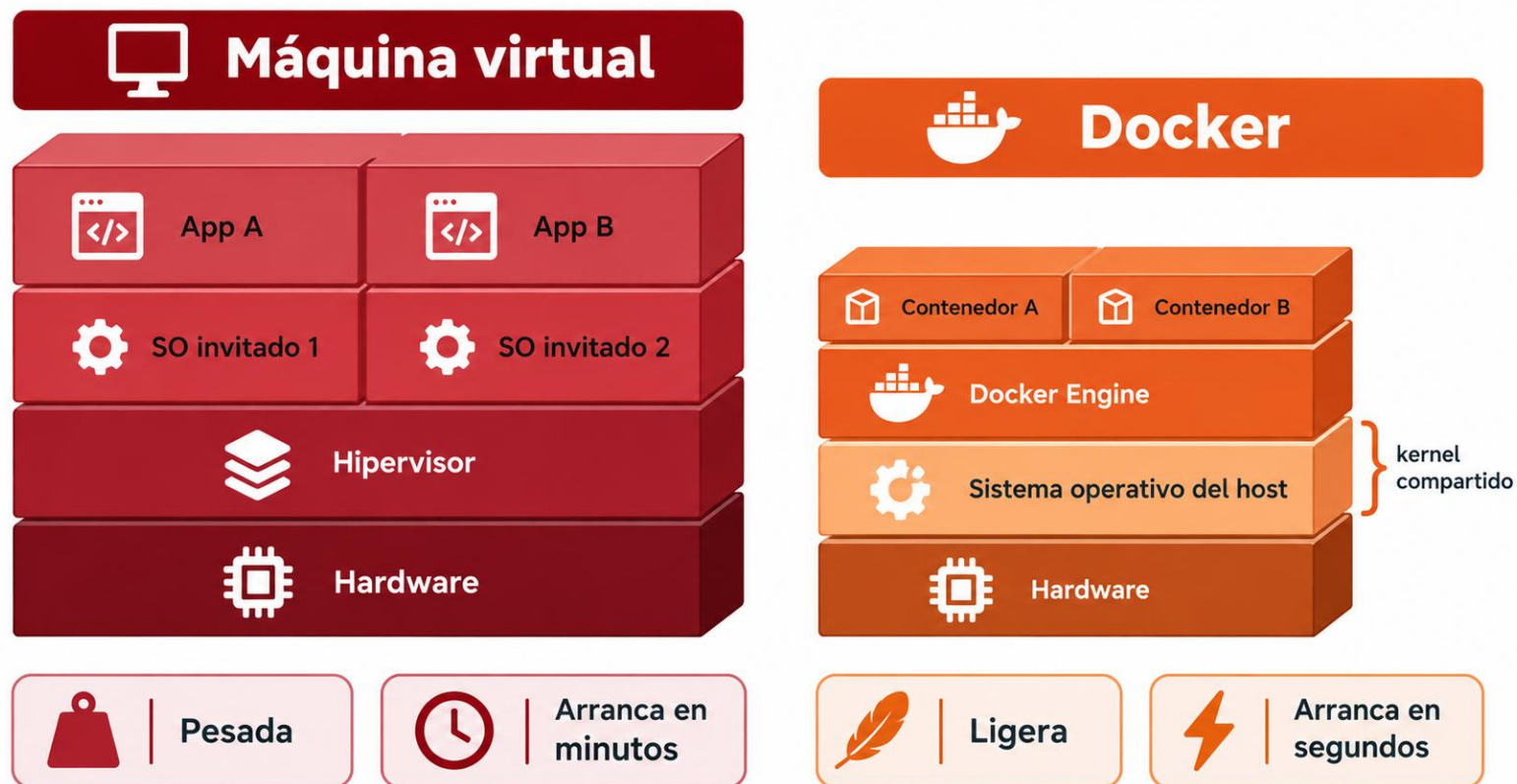
Igual en todas partes

Windows, macOS o Linux: dentro del contenedor el entorno es el mismo.

Metáfora: una caja estanca

Un contenedor es una caja sellada que lleva dentro exactamente lo que necesita para funcionar. Da igual qué versión de Python o de PostgreSQL tengas instalada fuera.

Docker vs. máquina virtual



Docker no sustituye a tu sistema operativo ni monta una máquina virtual por debajo: lo que sustituye es la costumbre de instalar el programa directamente en tu máquina.

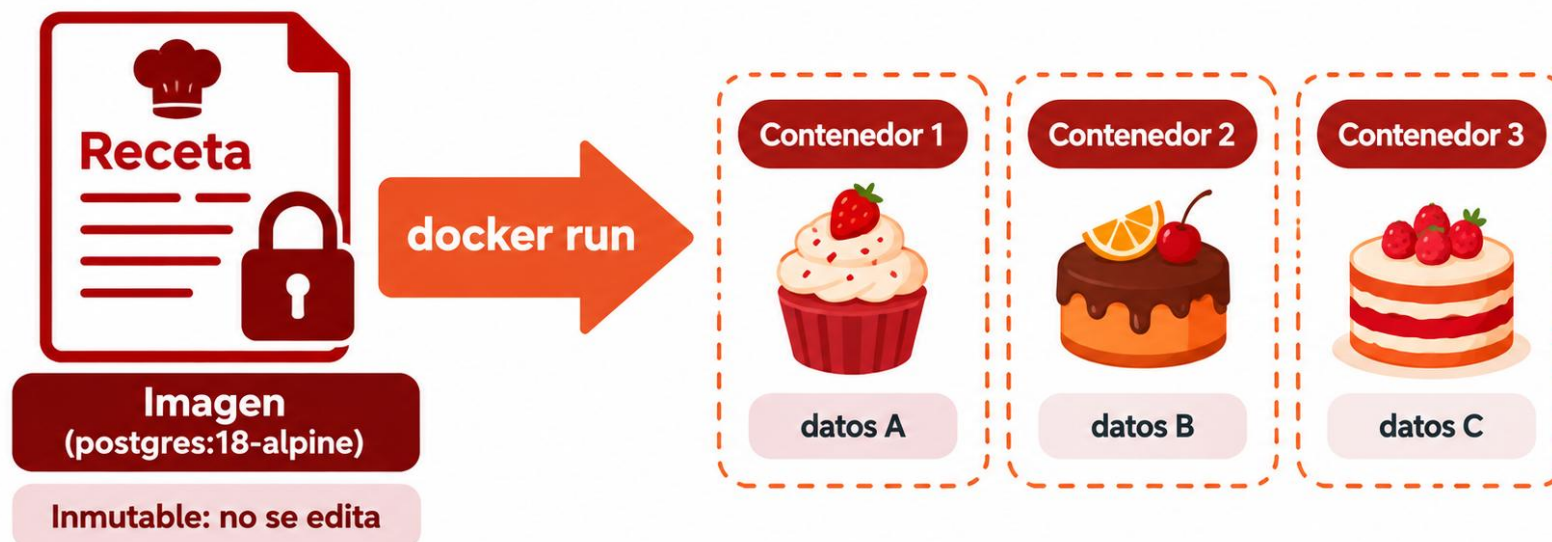
Imagen y contenedor

Imagen

Paquete de solo lectura con todo lo necesario para arrancar un programa. Es inmutable.
postgres:18-alpine = PostgreSQL 18 sobre Linux minimalista.

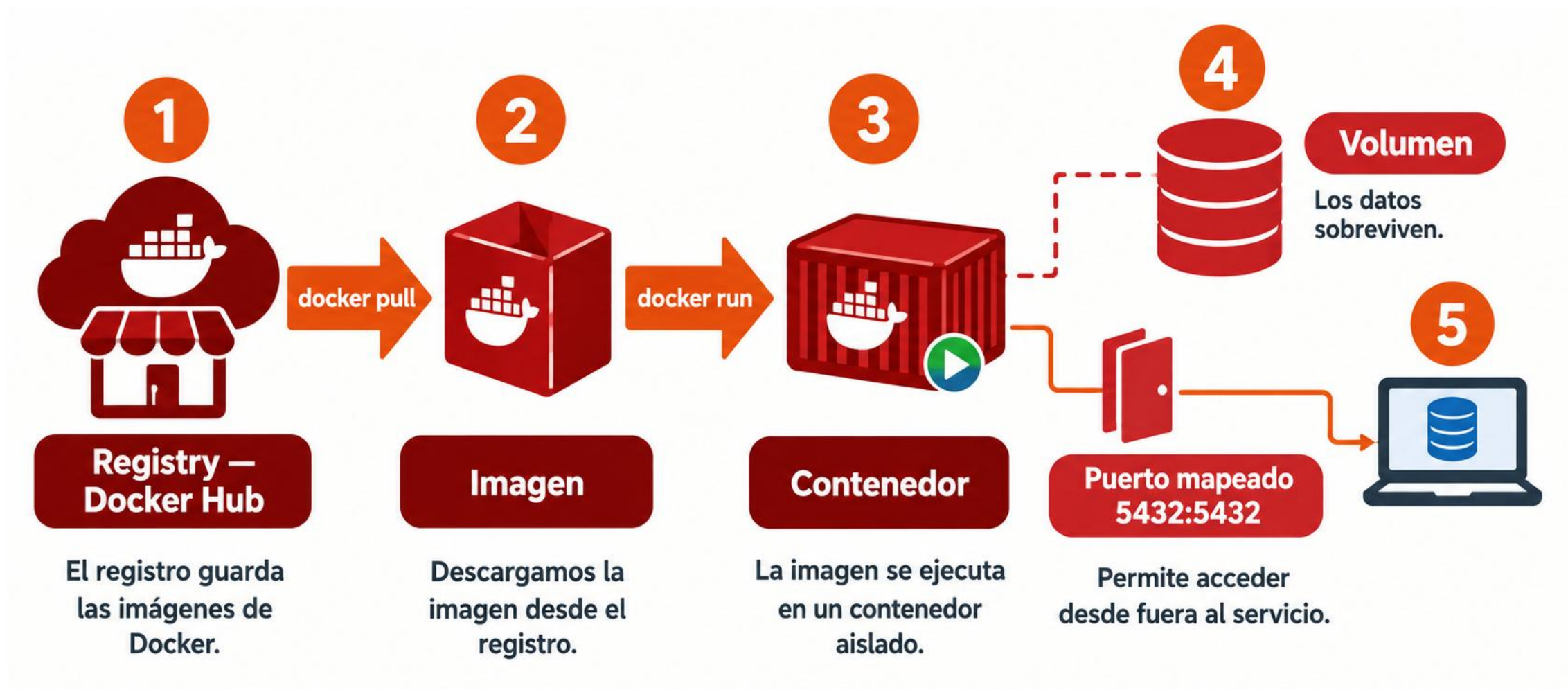
Contenedor

Una imagen puesta en marcha: proceso aislado, con su propio sistema de archivos y su propia red. De una imagen puedes arrancar varios.



➤ La misma receta, muchos pasteles independientes ➤

Los cinco conceptos, de un vistazo



Conceptos clave: Volumen

Un volumen es un espacio de almacenamiento que Docker gestiona por fuera del contenedor: sobrevive aunque el contenedor se borre y se vuelva a crear.

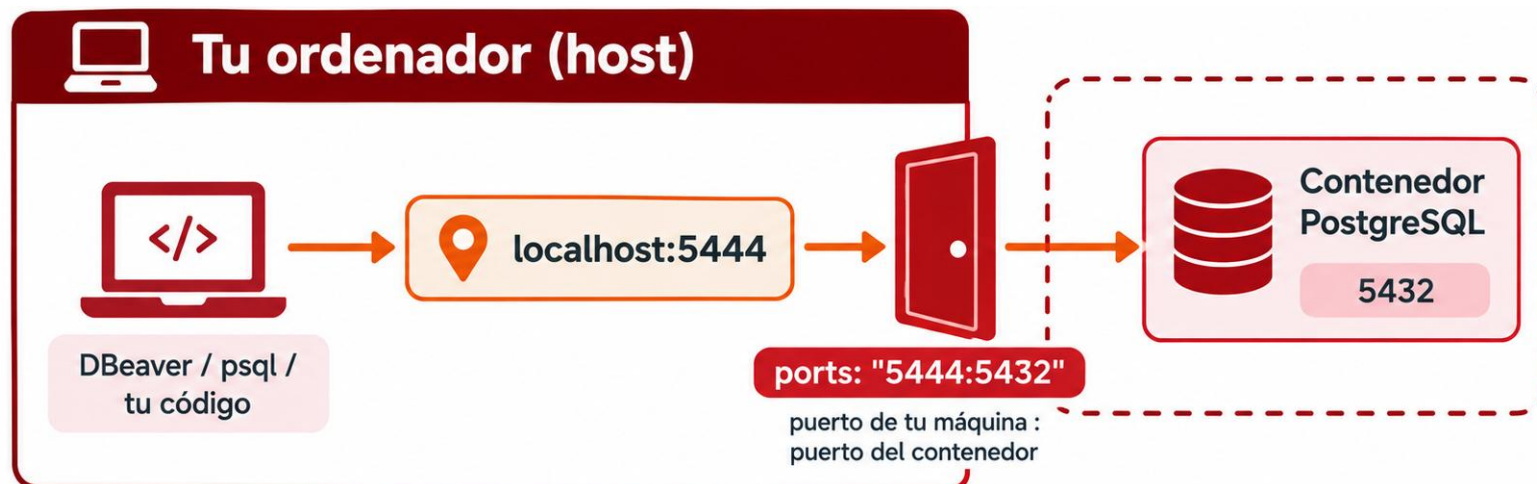


Registry y puerto mapeado

Registry: de dónde salen las imágenes

Un servidor donde se publican y descargan imágenes, parecido a una tienda de aplicaciones. Docker Hub es el registry público más usado (postgres, mongo, redis...).

Puerto mapeado: cómo entrar en algo aislado



Si el 5432 ya está ocupado en tu máquina, cambia el primer número.

Conceptos clave: resumen

Concepto	En una frase
Imagen	Plantilla inmutable de la que se crean contenedores.
Contenedor	Una imagen puesta en marcha; puede haber varios a la vez.
Registry	Servidor donde se publican y descargan imágenes (Docker Hub).
Volumen	Almacenamiento fuera del contenedor que sobrevive a su borrado.
Puerto mapeado	Puerta entre un puerto de tu máquina y uno del contenedor.

¿De dónde sale una imagen propia?

Además de descargar imágenes de un registry, Docker puede construir una imagen propia a partir de nuestros ficheros y de una receta llamada Dockerfile.



Construir no es ejecutar

`docker build` → crea una imagen
`docker run` → crea y ejecuta un contenedor

El resultado es reproducible

Si otra persona tiene los mismos ficheros y el Dockerfile, puede reconstruir la misma imagen siguiendo el mismo proceso.

Un Dockerfile mínimo

Un Dockerfile es un fichero de texto. Cada instrucción describe una decisión de construcción o de arranque.

```
FROM eclipse-temurin:21-jre

WORKDIR /app

COPY app.jar app.jar

EXPOSE 8080

ENTRYPOINT ["java", "-jar", "app.jar"]
```

FROM

elige la imagen de partida

WORKDIR

fija el directorio de trabajo

COPY

incorpora ficheros del proyecto

EXPOSE

documenta el puerto esperado

ENTRYPOINT / **CMD** indican qué proceso arrancará

Construir

`docker build -t mi-app:1.0 .`

.dockerignore

Excluye ficheros locales o sensibles del build.

Para ampliar: caché de construcción, builds multietapa y usuarios sin privilegios.

Comandos básicos de Docker

No hace falta memorizarlos, pero conviene reconocerlos:

Comando	Qué hace
docker pull <imagen>	Descarga una imagen desde un registry (Docker Hub por defecto).
docker images	Lista las imágenes descargadas en tu máquina.
docker run <imagen>	Crea y arranca un contenedor a partir de una imagen.
docker ps	Lista los contenedores en ejecución (-a incluye los detenidos).
docker stop <contenedor>	Detiene un contenedor en ejecución sin borrarlo.
docker rm <contenedor>	Elimina un contenedor detenido.
docker logs <contenedor>	Muestra la salida (logs) de un contenedor.

Docker Compose

Un proyecto real casi nunca necesita un único contenedor. Docker Compose describe todos los servicios en un fichero `docker-compose.yml` y los crea, conecta y arranca juntos con un solo comando.

Sin Compose

Un `docker run` por servicio, recordando puertos, variables de entorno y volúmenes de cada uno. No escala.

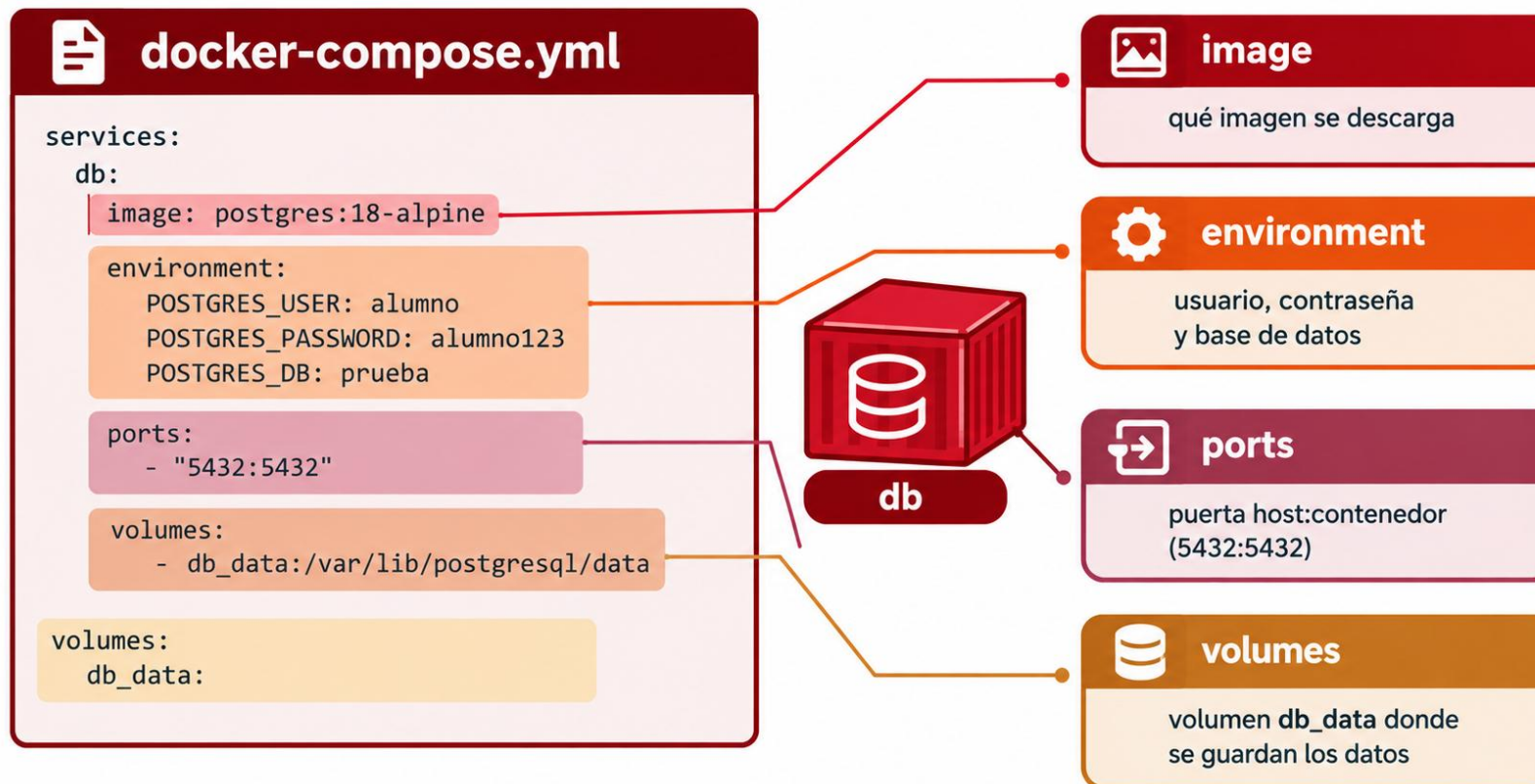
Con Compose

Un único fichero declarativo y un solo comando: `docker compose up -d`.

Cuando hay varios servicios

- Se dirigen unos a otros por red usando el nombre del servicio como si fuera una dirección.
- `depends_on` decide en qué orden arrancan. Lo practicarás en la Actividad 0.6, con un WordPress completo.

Un docker-compose.yml sencillo: una sola base de datos

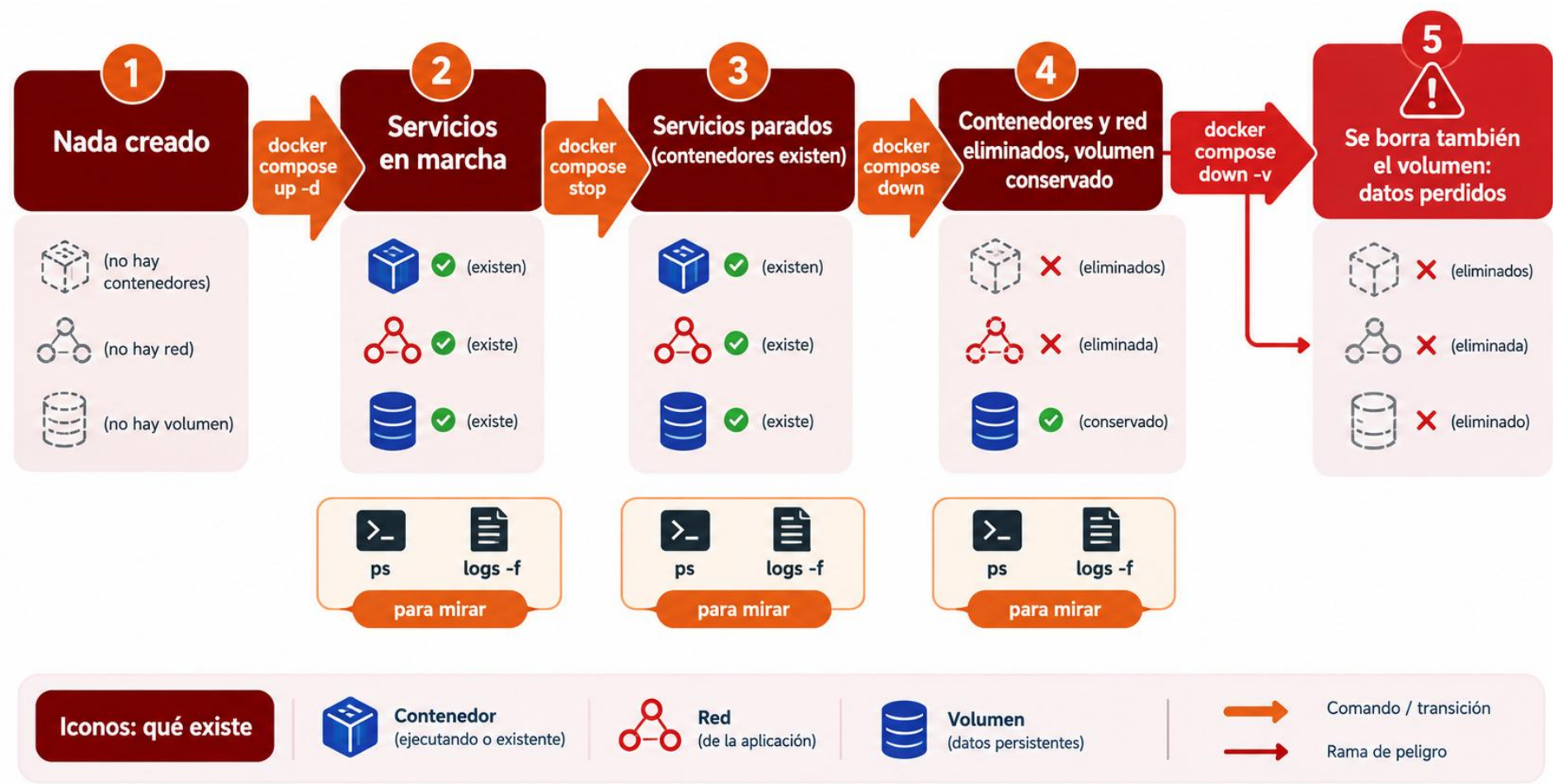


Si ya tienes PostgreSQL instalado en tu máquina escuchando en el 5432, este contenedor chocará con él: cambia el primer número, por ejemplo a "5444:5432".

Comandos básicos de Docker Compose

Comando	Qué hace
docker compose up -d	Crea y arranca todos los servicios definidos (-d en segundo plano).
docker compose ps	Lista los servicios del proyecto y su estado.
docker compose logs -f <servicio>	Muestra los logs de un servicio en tiempo real.
docker compose stop	Detiene todos los servicios sin eliminarlos.
docker compose down	Detiene y elimina contenedores y red. No borra volúmenes.
docker compose down -v	Igual que down, pero además elimina los volúmenes.

Ciclo de vida: cuidado con down -v



docker compose down -v borra los datos de verdad. Para un reinicio normal usa down (sin -v) o stop.

Ideas clave

1

Docker ejecuta aplicaciones en contenedores: aislados, ligeros, con el kernel del host compartido.

2

Una imagen es la plantilla inmutable; un contenedor es una instancia en ejecución de esa imagen.

3

Un volumen guarda los datos fuera del contenedor para que sobrevivan a su borrado.

4

Docker Compose describe varios contenedores en un `docker-compose.yml` y los levanta juntos.

5

`services`, `ports`, `volumes`, `environment` y `depends_on` son las claves esenciales del fichero.

6

`down` conserva los volúmenes; `down -v` también borra los datos.